Sequential Search and Binary Search

Data Structures Using C – Searching Algorithms

Department of Computer Science and Engineering

Graphic Era (Deemed to be University), Dehradun

What is Searching?

Searching is the process of locating a particular element in a collection.

Given an array:

$$A = [10,\ 20,\ 30,\ 40,\ 50]$$

and a value:

$$VAL = 30$$

we want to determine:

- Whether 30 exists in the array.
- The position or index of 30.

Here:

$$POS = 2$$

using zero-based indexing.

Common Searching Techniques

Two fundamental searching algorithms are:

Sequential Search

- Also called Linear Search.
- Checks elements one by one.
- Works on sorted and unsorted arrays.

Binary Search

- Uses a sorted array.
- Examines the middle element.
- Repeatedly halves the search interval.

What is Sequential Search?

Sequential Search, also called Linear Search, compares the required value with array elements one by one.

The search begins at the first element.
It continues until:

1. The required element is found, or
2. All elements have been examined.

Key Property

The array does not need to be sorted.

Linear Search: Basic Principle

Given:

$$A = [10,\ 8,\ 2,\ 7,\ 3]$$

Search:

$$VAL = 7$$

Compare sequentially:

$$10 \neq 7$$

$$8 \neq 7$$

$$2 \neq 7$$

$$7 = 7$$

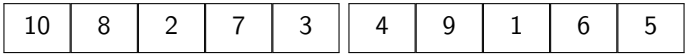
Textbook Example: Linear Search

Consider:

$$A = [10,\ 8,\ 2,\ 7,\ 3,\ 4,\ 9,\ 1,\ 6,\ 5]$$

Search for:

$$VAL = 7$$



The array is unsorted.
Therefore, Sequential Search is suitable.

Linear Search: Step 1

Current index:

$$i = 0$$

Current element:

$$A[0] = 10$$

Search value:

$$VAL = 7$$

Compare:

$$10 \neq 7$$

No match.
Move to the next index:

$$i = 1$$

Linear Search: Step 2

Current index:

$$i = 1$$

Current element:

$$A[1] = 8$$

Compare:

$$8 \neq 7$$

No match.

Move to:

$$i = 2$$

Linear Search: Step 3

Current index:

$$i = 2$$

Current element:

$$A[2] = 2$$

Compare:

$$2 \neq 7$$

No match.

Move to:

$$i = 3$$

Linear Search: Step 4

Current index:

$$i = 3$$

Current element:

$$A[3] = 7$$

Compare:

$$7 = 7$$

Search Successful

The value 7 is found.

$$\boxed{POS = 3}$$

Human-readable position: 4.

Linear Search: Complete Trace

Step	Index	Element	Result
1	0	10	No match
2	1	8	No match
3	2	2	No match
4	3	7	Match

Total comparisons:

$$\boxed{4}$$

Linear Search: Unsuccessful Case

Consider:

$$A = [10,\ 8,\ 2,\ 7,\ 3]$$

Search:

$$VAL = 15$$

Every element is compared with 15.
No match is found.

$$\boxed{POS = -1}$$

Total comparisons:

5

Linear Search: Algorithm

LINEAR_SEARCH(A, N, VAL)

1. Set $POS = -1$.
2. Set $i = 0$.
3. Repeat while $i < N$:
   1. If $A[i] = VAL$:
      - Set $POS = i$.
      - Stop searching.
   2. Otherwise increment i.
4. Return POS.

Linear Search: C Function

```c
int linearSearch(const int a[], int n, int key)
{
    for (int i = 0; i < n; i++)
    {
        if (a[i] == key)
            return i;
    }

    return -1;
}
```

Return Value

Returns the zero-based index if found.
Returns -1 if the element is absent.

Linear Search: Complete C Program

```c
#include <stdio.h>

int linearSearch(const int a[], int n, int key)
{
    for (int i = 0; i < n; i++)
    {
        if (a[i] == key)
            return i;
    }
    return -1;
}

int main(void)
{
    int a[] = {10, 8, 2, 7, 3, 4, 9, 1, 6, 5};
    int n = sizeof(a) / sizeof(a[0]);
    int key = 7;

    int pos = linearSearch(a, n, key);

    if (pos == -1)
        printf("Element not found\n");
    else
        printf("Found at index %d\n", pos);

    return 0;
}
```

Linear Search: Output

```
1 Found at index 3
```

Observation

The algorithm finds the first occurrence of the value.
It stops immediately after finding a match.

Linear Search: Best Case

The best case occurs when the value is the first array element.
Example:

$$A = [7,\ 10,\ 20,\ 30,\ 40]$$

Search:

$$VAL = 7$$

Only one comparison is required.

$$\boxed{T_{\text{best}}(n) = \Theta(1)}$$

Linear Search: Worst Case

The worst case occurs when:

- The value is the last element, or
- The value is not present.

Example:

$$A = [10,\ 20,\ 30,\ 40,\ 50]$$

Search:

$$VAL = 50$$

All five elements are examined.

$$\boxed{T_{\text{worst}}(n) = \Theta(n)}$$

Linear Search: Average Case

Assume the required value is present at any position with equal probability.

Expected comparisons:

$$E[C] = \frac{1 + 2 + \cdots + n}{n}$$

Using:

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

Therefore:

$$E[C] = \frac{n+1}{2}$$

$$\boxed{T_{\text{average}}(n) = \Theta(n)}$$

Linear Search: Complexity Summary

Case	Complexity
Best	$\Theta(1)$
Average	$\Theta(n)$
Worst	$\Theta(n)$
Auxiliary space	$O(1)$

Important

Linear Search works without requiring the array to be sorted.

Advantages and Limitations of Linear Search

Advantages

- Easy to understand.
- Easy to implement.
- Works on unsorted data.
- No preprocessing required.
- Constant auxiliary space.

Limitations

- Slow for large arrays.
- May examine every element.
- Does not exploit sorted order in its basic form.

What is Binary Search?

Binary Search is an efficient searching algorithm for a sorted array.

It repeatedly divides the search interval into two parts.

1. Find the middle element.
2. Compare the middle element with the key.
3. Continue in the appropriate half.
4. Repeat until found or the interval is empty.

Precondition

The array must be sorted according to the same ordering used for comparison.

Binary Search: Real-Life Analogy

Consider searching for a word in a dictionary.

Instead of checking every word:

1. Open the dictionary near the middle.
2. Compare the word with the target.
3. Decide whether to search earlier pages or later pages.
4. Repeat in the selected section.

Main Idea

Eliminate approximately half of the remaining search space at each step.

Binary Search: Important Variables

Variable	Meaning
BEG	First index of active interval
END	Last index of active interval
MID	Middle index
VAL	Value being searched
POS	Index of the matched value

Initially:

$$BEG = 0, \qquad END = n - 1$$

Middle index:

$$MID = BEG + \left\lfloor \frac{END - BEG}{2} \right\rfloor$$

Binary Search: Decision Rules

Compare:

$$VAL \quad \text{with} \quad A[MID]$$

Case 1: Match

If $VAL = A[MID]$, return MID.

Case 2: Search Left

If $VAL < A[MID]$, set $END = MID - 1$.

Case 3: Search Right

If $VAL > A[MID]$, set $BEG = MID + 1$.

Textbook Example: Binary Search

Consider:

$$A = [0,\ 1,\ 2,\ 3,\ 4,\ 5,\ 6,\ 7,\ 8,\ 9,\ 10]$$

Search:

$$VAL = 9$$

The array is already sorted.
Initially:

$$BEG = 0$$

$$END = 10$$

Binary Search: First Iteration

Initially:

$$BEG = 0, \qquad END = 10$$

Calculate:

$$MID = \left\lfloor \frac{0 + 10}{2} \right\rfloor = 5$$

Current element:

$$A[5] = 5$$

Compare:

$$9 > 5$$

Therefore, search the right half.

$$BEG = 6$$

Binary Search: Second Iteration

Current interval:

$$BEG = 6, \qquad END = 10$$

Calculate:

$$MID = \left\lfloor \frac{6 + 10}{2} \right\rfloor = 8$$

Current element:

$$A[8] = 8$$

Compare:

$$9 > 8$$

Therefore:

$$BEG = 9$$

Binary Search: Third Iteration

Current interval:

$$BEG = 9, \qquad END = 10$$

Calculate:

$$MID = \left\lfloor \frac{9 + 10}{2} \right\rfloor = 9$$

Current element:

$$A[9] = 9$$

Compare:

$$9 = 9$$

Search Successful

Binary Search: Complete Trace

Iteration	BEG	END	MID	A[MID]	Result
1	0	10	5	5	Search right
2	6	10	8	8	Search right
3	9	10	9	9	Found

Total middle-element comparisons:

$$\boxed{3}$$

Sequential Search would require 10 comparisons for this same array and key.

Binary Search: Unsuccessful Example

Consider:

$$A = [2,\ 4,\ 6,\ 8,\ 10,\ 12,\ 14]$$

Search:

$$VAL = 9$$

BEG	END	MID	A[MID]	Decision
0	6	3	8	Search right
4	6	5	12	Search left
4	4	4	10	Search left

Finally:

$$BEG = 4, \quad END = 3$$

Since $BEG > END$, the element is absent.

Binary Search: Algorithm

BINARY_SEARCH(A, N, VAL)

1. Set $BEG = 0$, $END = N - 1$.
2. Repeat while $BEG \leq END$:
   1. Calculate:
   $$MID = BEG + \left\lfloor \frac{END - BEG}{2} \right\rfloor$$
   2. If $A[MID] = VAL$, return MID.
   3. If $VAL < A[MID]$, set $END = MID - 1$.
   4. Otherwise, set $BEG = MID + 1$.
3. Return -1.

Binary Search: C Function

```c
int binarySearch(const int a[], int n, int key)
{
    int beg = 0;
    int end = n - 1;

    while (beg <= end)
    {
        int mid = beg + (end - beg) / 2;

        if (a[mid] == key)
            return mid;

        if (key < a[mid])
            end = mid - 1;
        else
            beg = mid + 1;
    }

    return -1;
}
```

Binary Search: Complete C Program

```c
#include <stdio.h>

int binarySearch(const int a[], int n, int key)
{
    int beg = 0, end = n - 1;

    while (beg <= end)
    {
        int mid = beg + (end - beg) / 2;

        if (a[mid] == key)
            return mid;
        else if (key < a[mid])
            end = mid - 1;
        else
            beg = mid + 1;
    }

    return -1;
}

int main(void)
{
    int a[] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    int n = sizeof(a) / sizeof(a[0]);
    int pos = binarySearch(a, n, 9);
```

Binary Search: Output

```
Index: 9
```

Important

Binary Search works correctly only when the array is sorted in the required order.

The presented implementation returns one matching index.
If duplicates exist, it does not necessarily return the first occurrence.

Binary Search: Best Case

The best case occurs when:

$$VAL = A[MID]$$

during the first comparison.

Only one middle-element comparison is required.

$$\boxed{T_{\text{best}}(n) = \Theta(1)}$$

Binary Search: Worst Case

At every iteration, approximately half of the remaining elements are eliminated.

The search interval sizes are:

$$n, \quad \frac{n}{2}, \quad \frac{n}{4}, \quad \frac{n}{8}, \ldots$$

After k halvings:

$$\frac{n}{2^k} \approx 1$$

Therefore:

$$k \approx \log_2 n$$

$$\boxed{T_{\text{worst}}(n) = \Theta(\log n)}$$

Binary Search: Complexity Summary

Case	Complexity
Best	$\Theta(1)$
Average	$\Theta(\log n)$
Worst	$\Theta(\log n)$
Auxiliary space, iterative	$O(1)$

Observation

Binary Search is significantly more efficient than Linear Search for large sorted arrays.

Linear Search vs Binary Search

Property	Linear Search	Binary Search
Sorted array required	No	Yes
Method	Sequential scan	Divide interval
Best time	$\Theta(1)$	$\Theta(1)$
Average time	$\Theta(n)$	$\Theta(\log n)$
Worst time	$\Theta(n)$	$\Theta(\log n)$
Iterative auxiliary space	$O(1)$	$O(1)$
Unsorted input	Supported	Not directly

Comparison: Number of Comparisons

Approximate worst-case comparisons:

n	Linear Search	Binary Search
10	10	4
100	100	7
1,000	1,000	10
10,000	10,000	14
100,000	100,000	17

Binary Search grows logarithmically, while Linear Search grows linearly.

Which Searching Algorithm Should We Use?

Choose Linear Search

- Data is unsorted.
- Collection is small.
- Only a few searches are needed.
- Simplicity is important.

Choose Binary Search

- Data is already sorted.
- Efficient random access is available.
- Collection is large.
- Many searches are required.

Common Programming Mistakes

1. Applying Binary Search to an unsorted array.
2. Using the wrong midpoint formula.
3. Forgetting to update BEG or END.
4. Writing $BEG < END$ instead of $BEG \leq END$ for this implementation.
5. Confusing zero-based index and one-based position.
6. Accessing elements outside array bounds.
7. Assuming Binary Search always finds the first duplicate.

Activity 1: Linear Search

Given:

$$A = [25,\ 10,\ 35,\ 5,\ 40,\ 15]$$

Search:

$$VAL = 40$$

Tasks:

1. Show each comparison.
2. Determine the index.
3. Count the total comparisons.
4. Identify the best-case input for the same search value.

Activity 1: Solution

Compare:

$$25 \neq 40$$

$$10 \neq 40$$

$$35 \neq 40$$

$$5 \neq 40$$

$$40 = 40$$

Result:

$$\boxed{POS = 4}$$

Total comparisons:

Activity 2: Binary Search

Given:

$$A = [3,\ 7,\ 11,\ 15,\ 19,\ 23,\ 27,\ 31,\ 35]$$

Search:

$$VAL = 27$$

Tasks:

1. Identify initial BEG and END.
2. Calculate MID.
3. Show every comparison.
4. Update the search interval.
5. Identify the final position.

Activity 2: Solution

BEG	END	MID	A[MID]	Decision
0	8	4	19	Search right
5	8	6	27	Found

Final result:

$$\boxed{POS = 6}$$

Total comparisons:

$$\boxed{2}$$

MCQ 1

Which searching algorithm can directly search an unsorted array?

1. Binary Search
2. Linear Search
3. Both always require sorted arrays
4. Neither

MCQ 1

Which searching algorithm can directly search an unsorted array?

1. Binary Search
2. Linear Search
3. Both always require sorted arrays
4. Neither

Answer

Option 2: Linear Search.

MCQ 2

What is the worst-case time complexity of Binary Search?

1. $O(1)$
2. $O(n)$
3. $O(\log n)$
4. $O(n^2)$

MCQ 2

What is the worst-case time complexity of Binary Search?

1. $O(1)$
2. $O(n)$
3. $O(\log n)$
4. $O(n^2)$

Answer

Option 3: $O(\log n)$.

MCQ 3

If $VAL < A[MID]$ in ascending-order Binary Search, what should happen?

1. $BEG = MID + 1$
2. $END = MID - 1$
3. $MID = MID + 1$
4. Stop immediately

MCQ 3

If $VAL < A[MID]$ in ascending-order Binary Search, what should happen?

1. $BEG = MID + 1$
2. $END = MID - 1$
3. $MID = MID + 1$
4. Stop immediately

Answer

Option 2: $END = MID - 1$.

MCQ 4

What does a return value of -1 usually indicate in the presented programs?

1. First element
2. Last element
3. Element not found
4. Invalid array size

MCQ 4

What does a return value of -1 usually indicate in the presented programs?

1. First element
2. Last element
3. Element not found
4. Invalid array size

Answer

Option 3: Element not found.

Practice Questions

1. Define searching.
2. Explain Sequential Search with an example.
3. Write a C program for Linear Search.
4. Explain Binary Search with an example.
5. Write the Binary Search algorithm.
6. Implement Binary Search in C.
7. Derive the worst-case complexity of Binary Search.
8. Compare Linear Search and Binary Search.
9. Explain why Binary Search requires sorted data.

Lecture Summary

Key Concepts

- Linear Search checks elements sequentially.
- It works on sorted and unsorted arrays.
- Linear Search worst case: $\Theta(n)$.
- Binary Search requires sorted input.
- It repeatedly halves the search interval.
- Binary Search worst case: $\Theta(\log n)$.
- Both iterative algorithms use $O(1)$ auxiliary space.

References

1. Reema Thareja, *Data Structures Using C*, Chapter 14: Searching and Sorting.
2. Ellis Horowitz and Sartaj Sahni, *Fundamentals of Data Structures*.
3. Thomas H. Cormen et al., *Introduction to Algorithms*.

Lecture Feedback

Your Feedback Matters

Please share your feedback on:

- Linear Search
- Binary Search
- Dry-run examples
- C implementation
- Complexity analysis



Scan to Submit Feedback

Thank You

Questions and Discussion

Sequential Search vs Binary Search